The script has two notable changes from the previous version in listing 5.28:

- The `if` clause is now compared against a value read from the `params` object (`params.gross_earnings_threshold`).
- The `gross_earnings_threshold` is set to `150` via the `params` block.

When the script is executed, Elasticsearch consults the `params` object and replaces the attribute with the value from the `params` object. If we want to change the value of gross earnings for setting a blockbuster flag (perhaps the `params.gross_earnings_threshold` needs to be updated to $500 million), we can simply pass the new value in the `params` flag.

Did you notice the hardcoded `params` value in the script? You may wonder why we hardcode the `gross_earnings_threshold`'s value in the script in the `params` object. Well, there is much more to scripting functionality than we've seen here. Scripts are compiled when they are executed for the first time. The compilation of the script carries a performance cost, so it is considered an expensive operation in Elasticsearch. However, scripts associated with dynamically changing parameters (using the `params` object) avoid this compilation cost because the script is compiled the first time only and is updated with the variable's (`params`) value when invoked for the rest of the time. This is a significant benefit, so the general practice is to provide dynamic variables via the `params` object to the script (refer to listing 5.29).

SCRIPTING LANGUAGES

The scripts developed in this chapter are derived from Elasticsearch's special scripting language, called Painless, to decode the logic and execute the scripts. The default language is Painless (we didn't specify the language explicitly in our code earlier). We can plug into other scripting languages (Mustache, Expression, or even Java) by using a `lang` parameter. Regardless of the language we use, we must follow this set pattern:

```
"script": {
    "lang": "painless|mustache|expression|java",
    "source": "...",
    "params": { ... }
  }
```

So far, we've updated individual documents using the `_update` API call or scripts. How about updating a bunch of documents that match a criterion? That's exactly what we learn in the next section.

5.4.4 Replacing documents

Let's say we need to replace an existing document with a new one. This is easy: we can use the same `PUT` request we performed earlier in the chapter when indexing a new document. Let's insert a new movie title (*Avatar*) in our `movies` document but associate it with an existing document (ID = 1).